

Description:

Create a database centric application using tiered architecture to generate and display a monthly report about total amount order amount placed by every customer of an e-commerce application.

Responsibility of Backend and Front-end:

The ASP.NET Core Web API application should be able to fetch name of every customer and the calculated total of order amount placed by the customer for the current month.

And the ASP.NET Core MVC Web Application should be able to display the output.

Note:

- i. If the customer has not placed any order for that month, then the 0 (zero) should be displayed as output along the customer's name.
- ii. Customer might place multiple orders in the same month. The application must calculate the total order amount for that month

A. Database:

Consider a database with two tables:

i. Customers: (schema)

Mobile_no (primary key, int/bigint, not null)

Name (not null, varchar (50))

Email_address (not null, varchar(max))

City (varchar (50), not null)

ii. Orders: (schema)

Order_id (auto generated, primary key, int, not null)

Order_date (datetime, not null)

Customer_mobile_no (int, not null, foreign key linking primary key from customers table)

Order_amount (decimal (18,2), default value- 0, not null)

Backend:

1. Create Asp.net core Web API application which will use code first approach to create the database tables
2. Use migration two create those tables
3. In the database put some sample data (put different order date for different months)
4. Create a controller which will have an action to return the necessary data

Front-end:

1. Create a ASP.NET Core MVC Web application.
2. The application should have a view with a dropdown list box which shows every month of a year. The use should be able to select a month and then will be able to view all the customer's name with total order amount placed for that month.

View Monthly Order Report:

Select a Month:

--select--
January
February
March
....

Name	Total Order Amount
Vinod	12000
Joydip	0
Arun	3400

Instructions:

1. Follow modular architecture
2. Create separate classes to represent entities and DTOs

3. **Following proper naming conventions**
4. **The application methods must be asynchronous**
5. **Create interfaces for loose coupling**
6. **Use dependency injection**
7. **All settings should be placed in JSON files and must be read into application**
8. **Use clean code**